For Python class or function, is it better to make argument default value as None or empty dictionary such as {}?

When defining default values for arguments in a Python class or function, it's generally recommended to use `None` instead of mutable default values like an empty dictionary (`{}`). This is due to the way Python handles default argument values.

Here's a brief explanation:

1. **Mutable Default Arguments (like `{}`)**: When you use a mutable default argument like an empty dictionary, Python creates it once when the function is defined, not each time the function is called. This means if the dictionary is modified (like adding a key-value pair) during one call, those changes will persist in subsequent calls. This can lead to unexpected behaviors and bugs.
2. **Immutable Default Arguments (like `None`)**: Using an immutable default argument, such as `None` , avoids this issue. Inside the function or method, you can check if the argument is `None` and, if so, initialize a new empty dictionary. This ensures that each call to the function gets a fresh dictionary.

Here’s an example to illustrate this:

python Copy code

```python
def example_function(arg=None):
    if arg is None:
        arg = {}
    # proceed with the function using the arg dictionary
```

This approach is safer and avoids the common pitfall of mutable default arguments.